

On-Policy Safe Reinforcement Learning Algorithms

A Mathematical Reference for the OmniSafe Implementation

OmniSafe Research Reference

Based on source code at [omnisafe/algorithms/on_policy/](https://github.com/omnisafe/algorithms/on_policy/)

2026

Contents

1 Background and Problem Formulation

1.1 Constrained Markov Decision Process

Safe reinforcement learning is formulated as a *Constrained Markov Decision Process* (CMDP) (?). A CMDP is a tuple $(\mathcal{S}, \mathcal{A}, P, r, c, \gamma, \mu_0)$ where:

- $\mathcal{S}$: state space; $\mathcal{A}$: action space;
- $P(s'|s, a)$: transition kernel;
- $r(s, a) \in \mathbb{R}$: reward function; $c(s, a) \in \mathbb{R}$: cost function;
- $\gamma \in (0, 1)$: discount factor; μ_0 : initial state distribution.

A stochastic policy $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ induces the *reward return* and *cost return*:

$$J^R(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad J^C(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t c(s_t, a_t) \right]. \quad (1)$$

The **safe policy optimization** problem is:

$$\max_{\pi} J^R(\pi) \quad \text{subject to} \quad J^C(\pi) \leq d, \quad (2)$$

where $d \geq 0$ is a pre-specified cost limit.

1.2 Value Functions and Advantage Estimation

For a policy π and signal $f \in \{r, c\}$, we define the *value function* and *action-value function*:

$$V_f^\pi(s) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t f(s_t, a_t) \mid s_0 = s \right], \quad Q_f^\pi(s, a) = f(s, a) + \gamma \mathbb{E}_{s'} [V_f^\pi(s')]. \quad (3)$$

The *advantage function* is $A_f^\pi(s, a) = Q_f^\pi(s, a) - V_f^\pi(s)$.

1.2.1 Generalized Advantage Estimation (GAE)

OmniSafe uses *Generalized Advantage Estimation* (?) to reduce variance in the policy gradient. Let $\delta_t^f = f(s_t, a_t) + \gamma V_f(s_{t+1}) - V_f(s_t)$ be the temporal-difference residual. The GAE estimate is:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{k=0}^{\infty} (\gamma \lambda)^k \delta_{t+k}^f, \quad (4)$$

where $\lambda \in [0, 1]$ interpolates between one-step TD ($\lambda = 0$, low variance, high bias) and Monte-Carlo returns ($\lambda = 1$, high variance, zero bias).

The corresponding *value target* is:

$$\hat{V}_f(s_t) = \hat{A}_t^{\text{GAE}} + V_f(s_t). \quad (5)$$

In OmniSafe, separate GAE estimates are computed for reward (λ_r) and cost (λ_c), yielding $\hat{A}_t^R$ and $\hat{A}_t^C$ respectively.

1.2.2 Importance Sampling

On-policy algorithms update the policy over multiple mini-batches after collecting data under the *old* policy π_{θ_k} . This requires an importance sampling ratio:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} = \exp(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_k}(a_t|s_t)). \quad (6)$$

1.3 Value Function Updates

All algorithms in OmniSafe update the reward and cost critics by minimizing the MSE loss:

$$\mathcal{L}_V = \frac{1}{N} \sum_{i=1}^N (\hat{V}_f(s_i) - V_f^\theta(s_i))^2 + \beta_{\text{norm}} \sum_j \|\theta_j\|^2, \quad (7)$$

where the optional L_2 regularisation term is controlled by `use_critic_norm` and `critic_norm_coef` β_{norm} .

2 Base Algorithms (No Explicit Safety Constraint)

2.1 Policy Gradient (PG)

Reference: ?.

The vanilla policy gradient maximises the expected return by ascending the gradient:

$$\nabla_\theta J^R = \mathbb{E}_{\tau \sim \pi_{\theta_k}} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \hat{A}_t^R \right]. \quad (8)$$

Using the importance-sampling rewrite, the surrogate loss minimised in OmniSafe is:

$$\mathcal{L}_{\text{PG}}(\theta) = -\mathbb{E}_t [\rho_t(\theta) \hat{A}_t^R]. \quad (9)$$

The advantage surrogate is simply $\hat{A}_t = \hat{A}_t^R$ (cost advantage unused). Gradients are clipped by ℓ_∞ -norm `max_grad_norm` before the Adam step. Updates are repeated for `update_iters` passes over mini-batches; early stopping occurs if $D_{\text{KL}}(\pi_{\theta_k} \parallel \pi_\theta) > \delta_{\text{KL}}$ (`target_kl`).

2.2 Proximal Policy Optimization (PPO)

Reference: ?.

PPO replaces the un-constrained policy gradient with a clipped surrogate to prevent destructively large updates:

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] - \beta_H \mathcal{H}[\pi_\theta], \quad (10)$$

where ϵ (`clip` = 0.2 by default) is the clipping radius and β_H (`entropy_coef`) weights the entropy bonus. The clipping operation enforces an implicit trust region: if the ratio deviates too far from 1, its gradient is zeroed.

Compared to PG, PPO inherits the same critic updates (Eq. ??) and the same advantage surrogate ($\hat{A}_t = \hat{A}_t^R$).

2.3 Natural Policy Gradient (NPG)

Reference: ?.

The natural gradient pre-conditions the ordinary gradient by the inverse Fisher information matrix $\mathbf{F}_\theta^{-1}$:

$$\tilde{\nabla}_\theta J^R = \mathbf{F}_\theta^{-1} \nabla_\theta J^R, \quad \mathbf{F}_\theta = \mathbb{E}_{\pi_{\theta_k}} [\nabla_\theta \log \pi_\theta (\nabla_\theta \log \pi_\theta)^\top]. \quad (11)$$

Computing $\mathbf{F}^{-1}$ directly is infeasible for neural networks. Instead, OmniSafe approximates the *Fisher-vector product* (FVP) $\mathbf{F}_\theta \mathbf{v}$ via automatic differentiation of the KL divergence:

$$\mathbf{F}_\theta \mathbf{v} = \nabla_\theta [(\nabla_\theta D_{\text{KL}}(\pi_{\theta_k} \parallel \pi_\theta))^\top \mathbf{v}] + \alpha_{\text{damp}} \mathbf{v}, \quad (12)$$

where α_{damp} (`cg_damping`) is a Tikhonov regulariser for numerical stability.

The natural gradient direction $\mathbf{x} = \mathbf{F}_\theta^{-1} \mathbf{g}$ (where $\mathbf{g} = -\nabla_\theta \mathcal{L}_{\text{PG}}$) is obtained by solving the linear system $\mathbf{F} \mathbf{x} = \mathbf{g}$ with the *conjugate gradient* (CG) algorithm (`cg_iters` = 15 iterations).

The step size α is set so that the KL divergence equals the trust-region budget:

$$\alpha = \sqrt{\frac{2\delta_{\text{KL}}}{\mathbf{x}^\top \mathbf{F} \mathbf{x}}}, \quad \theta_{\text{new}} = \theta_{\text{old}} + \alpha \mathbf{x}. \quad (13)$$

NPG does *not* perform line search; it takes the full step unconditionally.

Algorithm 1 Natural Policy Gradient (NPG) – actor update

Require: observations $\{s_i\}$, actions $\{a_i\}$, log-probs $\{\log \pi_{\theta_k}(a_i|s_i)\}$, advantages $\{\hat{A}_i^R\}$

- 1: Compute policy-gradient: $\mathbf{g} \leftarrow -\nabla_\theta \mathcal{L}_{\text{PG}}$
 - 2: Solve $\mathbf{F} \mathbf{x} = \mathbf{g}$ via CG using the FVP oracle (Eq. ??)
 - 3: Compute $\alpha \leftarrow \sqrt{2\delta_{\text{KL}}/(\mathbf{x}^\top \mathbf{F} \mathbf{x})}$
 - 4: $\theta \leftarrow \theta + \alpha \mathbf{x}$
-

2.4 Trust Region Policy Optimization (TRPO)

Reference: ?.

TRPO adds a backtracking line search to NPG to ensure both (i) actual improvement in the surrogate objective and (ii) satisfaction of the KL trust region:

$$\max_{\theta} \mathbb{E}_t [\rho_t \hat{A}_t^R] \quad \text{s.t.} \quad \mathbb{E}_s [D_{\text{KL}}(\pi_{\theta_k}(\cdot|s) \parallel \pi_\theta(\cdot|s))] \leq \delta_{\text{KL}}. \quad (14)$$

Starting from the NPG candidate $\theta' = \theta + \mathbf{x}$ (with the full NPG step), TRPO performs exponential back-off: $\theta'_j = \theta + \beta^j \alpha \mathbf{x}$ for $j = 0, 1, \dots, J$ (`update_iters` = 10, `decay` $\beta = 0.8$), accepting the first j for which:

1. $\Delta J^R = \mathcal{L}_{\text{before}} - \mathcal{L}(\theta'_j) > 0$ (surrogate improves), and
2. $D_{\text{KL}}(\pi_{\theta_k} \parallel \pi_{\theta'_j}) \leq \delta_{\text{KL}}$.

If no step satisfies both conditions, the policy is not updated.

3 Naïve Lagrangian Methods

3.1 The Lagrangian Relaxation

The CMDP (Eq. ??) can be converted to an unconstrained saddle-point problem by introducing a Lagrange multiplier $\lambda \geq 0$:

$$\max_{\pi} \min_{\lambda \geq 0} J^R(\pi) - \lambda (J^C(\pi) - d). \quad (15)$$

This gives the *penalised advantage surrogate* used by all naïve Lagrangian algorithms:

$$\hat{A}_t^{\text{Lag}} = \frac{\hat{A}_t^R - \lambda \hat{A}_t^C}{1 + \lambda}. \quad (16)$$

The normalisation by $(1 + \lambda)$ is an implementation detail that stabilises training.

3.2 Lagrange Multiplier Update

The multiplier λ is a differentiable parameter updated by gradient descent on the dual objective:

$$\mathcal{L}_\lambda = -\lambda (J_{\text{avg}}^C - d), \quad \lambda \leftarrow \text{clip}\left(\lambda - \eta_\lambda \frac{\partial \mathcal{L}_\lambda}{\partial \lambda}, 0, \lambda_{\text{max}}\right), \quad (17)$$

which amounts to $\lambda \leftarrow \text{clip}(\lambda + \eta_\lambda (J_{\text{avg}}^C - d), 0, \lambda_{\text{max}})$. Here η_λ (`lambda_lr`) is the multiplier learning rate and λ_{max} (`lagrangian_upper_bound`) is an optional upper bound.

3.3 PPO-Lag

PPO-Lag (?) substitutes the penalised advantage (Eq. ??) into the PPO clipped objective (Eq. ??):

$$\mathcal{L}_{\text{PPO-Lag}}(\theta) = -\mathbb{E}_t \left[\min\left(\rho_t \hat{A}_t^{\text{Lag}}, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t^{\text{Lag}}\right) \right]. \quad (18)$$

Before each policy update, λ is updated via Eq. ?? using the episodic cost average J_{avg}^C from the logger.

3.4 TRPO-Lag

TRPO-Lag uses the same penalised advantage $\hat{A}_t^{\text{Lag}}$ as the objective but employs the TRPO natural-gradient update with line search (Section ??) instead of the PPO clipped objective. The Lagrange update rule is identical to Eq. ??.

3.5 Reward Constrained Policy Optimization (RCPO)

Reference: ?.

RCPO is conceptually identical to TRPO-Lag but builds on the NPG base class, combining the natural gradient direction (no line search) with the penalised Lagrangian advantage:

$$\hat{A}_t^{\text{RCPO}} = \frac{\hat{A}_t^R - \lambda \hat{A}_t^C}{1 + \lambda}. \quad (19)$$

The multiplier update follows Eq. ??.

3.6 Primal-Dual Optimisation (PDO)

PDO (?) is the simplest Lagrangian baseline: a vanilla policy gradient with the penalised advantage $\hat{A}_t^{\text{Lag}}$ and the gradient-based multiplier update (Eq. ??). No clipping, natural gradient, or line search is applied.

4 Second-Order Constrained Methods

4.1 Constrained Policy Optimization (CPO)

Reference: ?.

CPO solves, at each iteration, the following constrained surrogate problem:

$$\max_{\theta} \mathbb{E}_t \left[\rho_t \hat{A}_t^R \right] \quad \text{s.t.} \quad \mathbb{E}_t \left[\rho_t \hat{A}_t^C \right] \leq c, \quad \mathbb{E}_s [D_{\text{KL}}(\pi_{\theta_k} \parallel \pi_{\theta})] \leq \delta_{\text{KL}}, \quad (20)$$

where $c = J_{\text{avg}}^C - d$ is the current constraint violation (signed).

4.1.1 Quadratic Approximation

Using the Fisher matrix $\mathbf{F}$ as a local metric and defining:

$$\mathbf{g} = \nabla_{\theta} \mathcal{L}_{\text{PG}} \quad (\text{reward gradient}), \quad (21)$$

$$\mathbf{b} = \nabla_{\theta} \mathcal{L}_{\text{cost}} \quad (\text{cost gradient}), \quad (22)$$

the problem (Eq. ??) is approximated as:

$$\max_{\Delta\theta} \mathbf{g}^{\top} \Delta\theta \quad \text{s.t.} \quad \mathbf{b}^{\top} \Delta\theta + c \leq 0, \quad \Delta\theta^{\top} \mathbf{F} \Delta\theta \leq 2\delta_{\text{KL}}. \quad (23)$$

Define:

$$\mathbf{x} = \mathbf{F}^{-1} \mathbf{g}, \quad \mathbf{p} = \mathbf{F}^{-1} \mathbf{b} \quad (\text{via CG}), \quad (24)$$

and the scalar quantities:

$$q = \mathbf{x}^{\top} \mathbf{F} \mathbf{x}, \quad r = \mathbf{g}^{\top} \mathbf{p}, \quad s = \mathbf{b}^{\top} \mathbf{p}, \quad A = q - \frac{r^2}{s}, \quad B = 2\delta_{\text{KL}} - \frac{c^2}{s}. \quad (25)$$

4.1.2 Analytical Solution by Case

The optimal dual variables (λ^*, ν^*) and the step direction depend on the geometry of the feasible region. OmniSafe implements four cases:

Case 4 ($\mathbf{b}^{\top} \mathbf{b} \leq \epsilon_b$ and $c < 0$): Cost gradient is negligible and current point is feasible. Use the plain TRPO step:

$$\Delta\theta = \alpha \mathbf{x}, \quad \alpha = \sqrt{2\delta_{\text{KL}}/q}, \quad \nu^* = 0, \quad \lambda^* = 1/\alpha. \quad (26)$$

Case 3 ($c < 0, B < 0$): The current point is feasible and the cost half-space does not intersect the trust region. Same as Case 4.

Cases 1 & 2 ($c \geq 0$ or $B \geq 0$): The QP has an active cost constraint. The dual objective is:

$$f_a(\lambda) = -\frac{1}{2} \left(\frac{A}{\lambda} + B\lambda \right) - \frac{rc}{s}, \quad f_b(\lambda) = -\frac{1}{2} \left(\frac{q}{\lambda} + 2\delta_{\text{KL}}\lambda \right). \quad (27)$$

The optimal λ^* is:

$$\lambda^* = (f_a(\lambda_a^*), f_b(\lambda_b^*)), \quad (28)$$

where $\lambda_a^* = \text{clip}(\sqrt{A/B}, \cdot)$ and $\lambda_b^* = \text{clip}(\sqrt{q/(2\delta_{\text{KL}})}, \cdot)$ are projected into their respective feasible intervals. Then:

$$\nu^* = \frac{\max(0, \lambda^* c - r)}{s}, \quad \Delta\theta = \frac{1}{\lambda^*} (\mathbf{x} - \nu^* \mathbf{p}). \quad (29)$$

Case 0 (full trust region infeasible): Fall back to purely decreasing cost:

$$\Delta\theta = -\nu^* \mathbf{p}, \quad \nu^* = \sqrt{2\delta_{\text{KL}}/s}. \quad (30)$$

4.1.3 CPO Line Search

After computing the analytical direction, CPO applies a backtracking line search with decay $\beta = 0.8$ (up to 20 steps) checking:

1. For Cases 3/4: surrogate reward improves.
2. $J^C(\theta') - J^C(\theta) \leq \max(0, c)$ (cost does not worsen beyond current violation).
3. $D_{\text{KL}}(\pi_{\theta_k} \| \pi_{\theta'}) \leq \delta_{\text{KL}}$.

4.2 Projection-Based Constrained Policy Optimization (PCPO)

Reference: ?.

PCPO simplifies CPO by replacing the case analysis with a direct two-step projection. The unconstrained natural gradient step is first computed, then projected onto the intersection of the KL trust region and the cost half-space:

$$\Delta\theta = \sqrt{\frac{2\delta_{\text{KL}}}{q}} \mathbf{F}^{-1} \mathbf{g} - \max\left(0, \frac{\sqrt{2\delta_{\text{KL}}/q} \cdot r + c}{s}\right) \mathbf{F}^{-1} \mathbf{b}. \quad (31)$$

This can be interpreted as: (i) scale the reward gradient to saturate the KL budget, then (ii) subtract a correction along the cost gradient to satisfy the linear constraint. PCPO inherits the CPO line search for final step acceptance.

5 First-Order Constrained Methods

5.1 First Order Constrained Optimisation in Policy Space (FOCOPS)

Reference: ?.

FOCOPS maintains a Lagrange multiplier λ and solves, in the space of non-parametric policies, a KL-penalised objective to find a locally-optimal constrained policy π^* , then projects it back to the parametric family.

5.1.1 Non-Parametric Optimal Policy

The closed-form solution of the non-parametric inner problem is:

$$\pi^*(a|s) \propto \pi_{\theta_k}(a|s) \exp\left(\frac{1}{\eta} (A_{\pi_{\theta_k}}^R(s, a) - \lambda A_{\pi_{\theta_k}}^C(s, a))\right), \quad (32)$$

where $\eta > 0$ (`focops_lam`) is a temperature parameter.

5.1.2 Parametric Projection Loss

The policy network is updated by minimising the KL distance to π^* , restricted to data points where the current KL from the old policy is within a threshold η' (`focops_eta`):

$$\mathcal{L}_{\text{FOCOPS}}(\theta) = \mathbb{E}_t \left[\left(D_{\text{KL}}(\pi_{\theta}(\cdot|s_t) \| \pi_{\theta_k}(\cdot|s_t)) - \frac{1}{\eta} \rho_t \hat{A}_t^{\text{Lag}} \right) \cdot \mathbf{1}[D_{\text{KL}}(\pi_{\theta} \| \pi_{\theta_k}) \leq \eta'] \right] - \beta_H \mathcal{H}[\pi_{\theta}], \quad (33)$$

where $\hat{A}_t^{\text{Lag}}$ is the penalised advantage from Eq. ???. The indicator masks updates for data points where the policy has already moved too far from π_{θ_k} , acting as a soft trust region.

The Lagrange multiplier is updated using Eq. ?? before the policy update.

5.2 Constrained Update Projection (CUP)

Reference: ?.

CUP is a two-stage algorithm. The first stage maximises the reward via a standard PPO update (Section ??). The second stage then projects the policy back towards constraint satisfaction via gradient descent on a cost-penalised KL loss.

5.2.1 Stage 1: Reward Update

Identical to PPO (Eq. ??) with the reward advantage $\hat{A}_t^R$.

5.2.2 Stage 2: Cost Projection

The second-stage loss penalises cost advantage while regularising by KL divergence from the pre-update policy π_{θ_0} (before Stage 1):

$$\mathcal{L}_{\text{CUP-cost}}(\theta) = \mathbb{E}_t \left[\lambda \frac{1 - \gamma^\nu}{1 - \gamma} \rho_t \hat{A}_t^C + D_{\text{KL}}(\pi_\theta(\cdot|s_t) \parallel \pi_{\theta_0}(\cdot|s_t)) \right], \quad (34)$$

where $\nu = \lambda_c$ is the cost GAE parameter and $\frac{1 - \gamma^\nu}{1 - \gamma}$ is a normalisation constant derived from the discounted cost return identity. The KL term prevents the cost projection from moving too far from the reward-updated policy.

Updates iterate for `update_iters` steps with early KL stopping.

6 PID Lagrangian Methods

6.1 PID Controller for the Lagrange Multiplier

Reference: ?.

The naïve gradient update of λ (Eq. ??) can be slow to respond to sudden constraint violations. PID-Lagrangian replaces it with a Proportional-Integral-Derivative controller that uses the error signal $e_t = J_{\text{avg}}^C - d$:

$$\text{P term: } \tilde{e}_t = \alpha_P \tilde{e}_{t-1} + (1 - \alpha_P) e_t, \quad (35)$$

$$\text{I term: } I_t = \max(0, I_{t-1} + K_i e_t), \quad (36)$$

$$\text{D term: } \tilde{C}_t = \alpha_D \tilde{C}_{t-1} + (1 - \alpha_D) J_{\text{avg}}^C, \quad D_t = \max(0, \tilde{C}_t - \tilde{C}_{t-\tau}), \quad (37)$$

$$\text{Output: } \lambda_t = \max(0, K_p \tilde{e}_t + I_t + K_d D_t), \quad (38)$$

where α_P, α_D are EMA smoothing coefficients (`pid_delta_p_ema_alpha`, `pid_delta_d_ema_alpha`), K_p, K_i, K_d are the PID gains, and τ (`pid_d_delay`) is the delay for the derivative window. The integral is initialised to `lagrangian_multiplier_init` and is clamped to zero from below.

6.2 TRPO-PID

TRPO-PID combines TRPO (Section ??) with the PID multiplier (Eqs. ??-??). The penalised advantage is:

$$\hat{A}_t^{\text{TRPO-PID}} = \frac{\hat{A}_t^R - \lambda_t \hat{A}_t^C}{1 + \lambda_t}. \quad (39)$$

6.3 CPPOPID

CPPOPID replaces the TRPO update with a PPO clipped objective while retaining the PID multiplier:

$$\mathcal{L}_{\text{CPPO-PID}}(\theta) = -\mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t^{\text{PID}}, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t^{\text{PID}} \right) \right], \quad \hat{A}_t^{\text{PID}} = \frac{\hat{A}_t^R - \lambda_t \hat{A}_t^C}{1 + \lambda_t}. \quad (40)$$

7 Penalty Function Methods

7.1 Penalised Proximal Policy Optimisation (P3O)

Reference: ?.

P3O adds a direct ReLU penalty on the cost surrogate to the PPO objective:

$$\mathcal{L}_{\text{P3O}}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \kappa \text{ReLU} \left(\mathbb{E}_t \left[\rho_t \hat{A}_t^C \right] + c \right), \quad (41)$$

where $c = J_{\text{avg}}^C - d$ is the current signed violation and κ (**kappa**) is a fixed penalty coefficient. Unlike Lagrangian methods, the penalty coefficient κ is not adapted; it is a fixed hyperparameter. The objective is a single loss combining reward and cost in one backward pass.

7.2 Interior-Point Policy Optimisation (IPO)

Reference: ?.

IPO uses an interior-point (log-barrier) inspired dynamic penalty that diverges as the cost approaches the limit:

$$\kappa_t = \frac{\kappa_0}{d - J_{\text{avg}}^C + \epsilon}, \quad \kappa_t \leftarrow \min(\kappa_t, \kappa_{\text{max}}), \quad (42)$$

where κ_0 (**kappa**) scales the penalty, ϵ is a small constant for numerical stability, and κ_{max} (**penalty max**) caps the penalty to prevent training instability. The penalised advantage is:

$$\hat{A}_t^{\text{IPO}} = \frac{\hat{A}_t^R - \kappa_t \hat{A}_t^C}{1 + \kappa_t}, \quad (43)$$

inserted into the standard PPO objective. The penalty automatically increases as J^C approaches d , providing interior-point-like barrier behaviour without requiring dual variable optimisation.

8 Primal Feasibility Method

8.1 Constrained Reward Policy Optimisation (OnCRPO)

Reference: ?.

CRPO avoids the Lagrangian entirely and switches between two objectives based on the current constraint satisfaction:

$$\hat{A}_t^{\text{CRPO}} = \begin{cases} \hat{A}_t^R & \text{if } J_{\text{avg}}^C \leq d + \Delta, \\ -\hat{A}_t^C & \text{otherwise,} \end{cases} \quad (44)$$

where Δ (**distance**) is a tolerance slack. When the constraint is satisfied, the algorithm performs a standard reward-maximisation step; when violated, it performs a cost-reduction step. The TRPO update (natural gradient + line search) is applied in both cases. This primal approach provably converges under mild assumptions (?).

9 State Augmentation Methods

State augmentation methods modify the *environment* rather than the policy update, encoding safety information directly into the observation. Standard PG algorithms (PPO, TRPO) are then applied without modification.

9.1 Saute RL (PPO-Saute, TRPO-Saute)

Reference: ?.

Saute augments the state with a running *safety budget* z_t :

$$\tilde{s}_t = (s_t, z_t), \quad z_{t+1} = \frac{z_t - c(s_t, a_t)}{\gamma}, \quad (45)$$

with $z_0 = d/(1 - \gamma)$ (the per-step budget). The reward is replaced by a safe reward:

$$\tilde{r}(s_t, a_t, z_t) = \begin{cases} r(s_t, a_t) & \text{if } z_t > 0, \\ r_{\text{unsafe}} & \text{otherwise.} \end{cases} \quad (46)$$

Since the constraint information is embedded in the state, standard unconstrained PPO/TRPO can be applied to the augmented MDP $(\mathcal{S} \times \mathbb{R}, \mathcal{A}, \tilde{P}, \tilde{r}, \gamma, \tilde{\mu}_0)$.

9.2 Simmer RL (PPO-Simmer-PID, TRPO-Simmer-PID)

Reference: ?.

Simmer extends Saute by actively controlling the safety budget z_t via a PID controller. At each epoch, the observed cost J_{avg}^C is fed into a budget controller that adjusts the initial budget z_0 for the next epoch:

$$z_0^{(k+1)} = z_0^{(k)} + \text{PID}(J_{\text{avg}}^C{}^{(k)} - d). \quad (47)$$

The PID controller has the same structure as in Eqs. ??–??. but it modulates the safety budget rather than a penalty multiplier. Within each episode, the Saute state augmentation is used unchanged.

10 Early Termination

10.1 PPO-EarlyTerminated and TRPO-EarlyTerminated

Reference: ?.

Early termination is the simplest safety mechanism: episodes are forcibly terminated when the cumulative cost within an episode exceeds a per-episode threshold d_{ep} . Formally, the *effective horizon* is:

$$T^*(\tau) = \min \left(T, \inf \left\{ t : \sum_{k=0}^t c(s_k, a_k) > d_{\text{ep}} \right\} \right). \quad (48)$$

All returns are computed only up to T^* , so the policy naturally learns to avoid constraint violations that cut trajectories short. PPO or TRPO is then applied to the truncated trajectories without any additional constraint-handling modification.

11 Summary and Comparison

Table 1: Summary of on-policy safe RL algorithms in OmniSafe.

Algorithm	Category	Base Update	Constraint Handling	Multiplier
PG	Unconstrained	1st-order PG	—	—
PPO	Unconstrained	Clipped PG	—	—
NPG	Unconstrained	Natural PG	—	—
TRPO	Unconstrained	Natural PG + LS	—	—
PDO	Naïve Lagrange	1st-order PG	Penalised adv.	Gradient
PPO-Lag	Naïve Lagrange	Clipped PG	Penalised adv.	Gradient
TRPO-Lag	Naïve Lagrange	Natural PG + LS	Penalised adv.	Gradient
RCPO	Naïve Lagrange	Natural PG	Penalised adv.	Gradient
CPO	2nd-order	Natural PG + LS	Dual QP (4 cases)	Analytical
PCPO	2nd-order	Natural PG + LS	Projection	Analytical
FOCOPS	1st-order	Gradient + KL clip	KL-reg. penalised adv.	Gradient
CUP	1st-order	Clipped PG + cost step	Two-stage KL projection	Gradient
TRPO-PID	PID Lagrange	Natural PG + LS	Penalised adv.	PID
CPPOPID	PID Lagrange	Clipped PG	Penalised adv.	PID
P3O	Penalty	Clipped PG	ReLU penalty	Fixed
IPO	Penalty	Clipped PG	Log-barrier penalty	Dynamic
OnCRPO	Primal	Natural PG + LS	Objective switching	None
PPO-Saute	State Aug.	Clipped PG	Augmented state	None
TRPO-Saute	State Aug.	Natural PG + LS	Augmented state	None
PPO-Simmer-PID	State Aug.	Clipped PG	Aug. state + PID budget	PID
TRPO-Simmer-PID	State Aug.	Natural PG + LS	Aug. state + PID budget	PID
PPO-EarlyTerm	Early Term.	Clipped PG	Episode termination	None
TRPO-EarlyTerm	Early Term.	Natural PG + LS	Episode termination	None

Key design axes. The algorithms differ along four orthogonal axes:

1. **Policy update:** first-order (PG, PPO), or second-order natural gradient (NPG, TRPO) which provides better credit assignment at the cost of Fisher-vector-product computation.
2. **Constraint enforcement:** Lagrangian relaxation, explicit quadratic programming (CPO), penalty functions, state augmentation, or primal feasibility switching (CRPO).
3. **Multiplier adaptation:** fixed, gradient-based dual ascent, or PID control.

4. **Environment modification:** standard CMDP or modified (Saute, Simmer, early termination).

12 Shared Implementation Details

12.1 Rollout and Data Collection

All on-policy algorithms collect T steps per epoch using `OnPolicyAdapter.rollout`, storing $(s_t, a_t, r_t, c_t, V^R(s_t), V^C(s_t), \log \pi_{\theta_k}(a_t|s_t))$ in the `VectorOnPolicyBuffer`. At the end of each rollout, `finish_path` is called to compute GAE advantages (Eq. ??) and value targets.

12.2 Actor Network

The policy is parameterised as a Gaussian distribution $\pi_{\theta}(a|s) = \mathcal{N}(\mu_{\theta}(s), \text{diag}(\sigma_{\theta}^2))$ with separate mean and log-standard-deviation heads. For discrete action spaces, a categorical distribution is used instead.

12.3 Distributed Training

OmniSafe uses MPI-style distributed training where gradients and statistics are averaged across workers via `distributed.avg_grads` and `distributed.dist_avg`.

12.4 Hyperparameter Defaults

Table 2: Common hyperparameter defaults in OmniSafe on-policy algorithms.

Parameter	Symbol	Default
Steps per epoch	T	20 000
Discount factor	γ	0.99
GAE λ (reward)	λ_r	0.95
GAE λ (cost)	λ_c	0.95
Trust region budget	δ_{KL}	0.02 (PG/PPO), 0.01 (TRPO/CPO)
PPO clip radius	ϵ	0.2
Critic update iterations	—	40
Mini-batch size	—	64
Gradient norm clip	—	40.0
CG iterations	—	15
CG damping	α_{damp}	0.1
Lagrange LR	η_{λ}	varies
Cost limit	d	25

References

Achiam, J., Held, D., Tamar, A., & Abbeel, P. (2017). Constrained policy optimization. In *International Conference on Machine Learning (ICML)*, pp. 22–31.

- Altman, E. (1999). *Constrained Markov Decision Processes*. CRC Press.
- Chow, Y., Ghavamzadeh, M., Janson, L., & Pavone, M. (2017). Risk-constrained reinforcement learning with percentile risk criteria. *Journal of Machine Learning Research*, 18(167), 1–51.
- Ji, J., Zhang, B., Zhou, J., Pan, X., Dai, J., Li, P., Chen, J., Zheng, B., Liu, Y., & Yang, Y. (2023). OmniSafe: An infrastructure for accelerating safe reinforcement learning research. *arXiv preprint arXiv:2305.09304*.
- Kakade, S. (2001). A natural policy gradient. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1531–1538.
- Liu, Y., Ding, J., & Liu, X. (2020). IPO: Interior-point policy optimization under constraints. In *AAAI Conference on Artificial Intelligence*.
- Liu, Z., Cen, Z., Isenbaev, V., Liu, W., Wu, S., Li, B., & Zhao, D. (2022). Constrained variational policy optimization for safe reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- Ray, A., Achiam, J., & Amodei, D. (2019). Benchmarking safe exploration in deep reinforcement learning. *arXiv preprint arXiv:1910.01708*.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2015). High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*.
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). Trust region policy optimization. In *International Conference on Machine Learning (ICML)*, pp. 1889–1897.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Sootla, A., Cowen-Rivers, A. I., Jafferjee, T., Wang, Z., Mguni, D. H., Wang, J., & Ammar, H. (2022). Sauté RL: Almost surely safe reinforcement learning using state augmentation. In *International Conference on Machine Learning (ICML)*.
- Sootla, A., Cowen-Rivers, A. I., Wang, J., & Ammar, H. B. (2022). SIMMER: Adaptive budget scheduling for safe reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Stooke, A., Achiam, J., & Abbeel, P. (2020). Responsive safety in reinforcement learning by PID Lagrangian methods. In *International Conference on Machine Learning (ICML)*.
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1057–1063.
- Tessler, C., Mankowitz, D. J., & Mannor, S. (2018). Reward constrained policy optimization. *arXiv preprint arXiv:1805.11074*.

- Xu, T., Liang, Y., & Lan, G. (2021). CRPO: A new approach for safe reinforcement learning with convergence guarantee. In *International Conference on Machine Learning (ICML)*.
- Yang, T.-Y., Rosca, J., Narasimhan, K., & Ramadge, P. J. (2020). Projection-based constrained policy optimization. In *International Conference on Learning Representations (ICLR)*.
- Yang, L., Ji, J., Dai, J., Zhang, L., Zhou, B., Li, P., Yang, Y., & Pan, G. (2022). Constrained update projection approach to safe policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zhang, Y., Vuong, Q., & Ross, K. W. (2020). First order constrained optimization in policy space. In *Advances in Neural Information Processing Systems (NeurIPS)*.